There are additional differences between MVC and Razor Pages, which I have highlighted throughout the book, but this layout difference is really the biggest win. Razor Pages embraces the fact that you're building a page-based application and optimizes your workflow by keeping everything related to a single page together.

TIP You can think of each Razor Page as a mini controller focused on a single page. Page handlers are functionally equivalent to MVC controller action methods.

This layout also has the benefit of making each page a separate class. This contrasts with the MVC approach of making each page an action on a given controller. Each Razor Page is cohesive for a particular feature, such as displaying a to-do item. MVC controllers contain action methods that handle multiple different features for a more abstract concept, such as all the features related to to-do items.

NOTE ASP.NET Core is eminently customizable, so you don't have to group your MVC applications by type; it's simply the default state and the easy path. In fact, if you do choose to use MVC controllers, I strongly suggest grouping using feature folders instead. This MSDN article provides a good introduction: http://mng.bz/mVOr.

Another important point is that Razor Pages doesn't lose any of the separation of concerns that MVC has. The view part of Razor Pages is still concerned only with rendering HTML, and the handler is the coordinator that calls out to the application model. The only real difference is the lack of the explicit view model that you have in MVC, but it's perfectly possible to emulate this in Razor Pages if that's a deal-breaker for you.

The benefits of using Razor Pages are particularly noticeable when you have content websites, such as marketing websites, where you're mostly